PropOS Flutter 专家级开发指南

元信息	值
版本	v1.0
日期	2026-04-20
适用对象	前端开发者（中高级）
技术栈	Flutter 3.x · Dart 3 · flutter*bloc ^8.x · go*router ^14.x · dio ^5.x · get_it ^7.x · freezed
前置知识	Dart 3（sealed class / pattern matching）· BLoC 模式 · Clean Architecture
项目目录	`flutter_app/`

目录

一、架构总览

1.1 系统定位

1.2 架构层次图

1.3 核心原则

二、环境搭建与工程配置

2.1 前置工具链

2.2 本地开发启动

2.3 关键配置文件

2.4 核心依赖清单

三、目录结构详解

3.1 命名规范

四、核心分层架构与数据流

4.1 严格单向数据流

4.2 创建完整功能模块的文件依赖顺序

五、API 层开发规范

5.1 HTTP 客户端（api_client.dart）

5.2 响应信封解析

5.3 Repository 实现范式

5.4 禁止事项

六、BLoC / Cubit 状态管理规范

6.1 State 四态（必须使用 freezed sealed union）

6.2 Cubit 标准模板
6.3 何时选 Cubit vs Bloc
6.4 Bloc 事件驱动示例
6.5 禁止事项
七、Page / Widget 层开发规范
7.1 Page 标准结构
7.2 状态渲染规则
7.3 Page 与 Widget 职责边界
7.4 文件复杂度控制
7.5 日期显示规则
八、实体与 DTO 类型体系
8.1 Domain Entity（纯 Dart）
8.2 Data Model（freezed DTO）
8.3 类型规则
九、路由与导航（go_router）
9.1 路由表定义
9.2 路由路径常量
9.3 导航方式
9.4 路由守卫
十、依赖注入（get_it）
10.1 注册模式
10.2 注册规则
十一、UI 规范与 Material 3 设计体系
11.1 ThemeData 配置
11.2 语义色扩展
11.3 状态色语义映射
11.4 颜色使用禁令
11.5 状态标签 Widget
十二、Mock 数据层规则
12.1 控制开关
12.2 Mock 拦截器
12.3 规则
十三、常量管理
十四、错误处理体系
14.1 三层错误处理链
14.2 ApiException 定义

14.3 BLoC 层统一 catch
十五、平台适配（iOS / Android / HarmonyOS Next）
15.1 三平台策略
15.2 平台判断
15.3 Safe Area
十六、测试策略
16.1 测试金字塔
16.2 BLoC 测试范式
十七、性能优化
十八、新功能模块开发 SOP
Step 1：Domain 层
Step 2：Data 层
Step 3：Presentation 层
Step 4：注册集成
Step 5：Mock + 测试
十九、常见问题与排错
Q1：freezed 代码生成报错
Q2：get_it 注册找不到实例
Q3：BlocBuilder 不刷新
Q4：go_router 路由不匹配
Q5：HarmonyOS 编译失败
二十、附录
附录 A：完整依赖清单（pubspec.yaml 节选）
附录 B：BLoC / Cubit 清单
附录 C：与 Admin 端对照

一、架构总览

1.1 系统定位

PropOS（Property Operating System）移动端基于 **Flutter** 构建，一套代码覆盖三个移动平台：

平台	运行时	说明
iOS	App	运营人员移动办公主力端
Android	App	运营人员移动办公主力端

HarmonyOS Next	App	鸿蒙原生兼容（Phase 1 必支持）

> Flutter Web 不在当前交付范围内。PC 管理后台由独立 `admin/` （Vue 3 + Element Plus）承载。

1.2 架构层次图

```
┌──────────────────────────────────────────────────────────┐
│                    Page / Widget（UI 层）                 │
│     BlocBuilder / BlocListener ← 只读取 State，不含业务逻辑 │
├──────────────────────────────────────────────────────────┤
│                   BLoC / Cubit（状态管理层）              │
│          注入 domain Repository 接口，管理 freezed 四态 State │
├──────────────────────────────────────────────────────────┤
│                  Domain（纯 Dart 业务逻辑层）             │
│          entities/ + repositories/(abstract) + usecases/  │
│          无 Flutter SDK 依赖，可独立单元测试               │
├──────────────────────────────────────────────────────────┤
│                   Data（数据实现层）                      │
│          models/(freezed DTO) + repositories/(实现)       │
│          调用 ApiClient，实现 domain 层抽象接口            │
├──────────────────────────────────────────────────────────┤
│                    ApiClient（dio 封装）                  │
│          JWT 注入 · 信封解析 · Token 刷新 · 错误统一转换    │
├──────────────────────────────────────────────────────────┤
│                    Backend REST API                      │
└──────────────────────────────────────────────────────────┘
```

1.3 核心原则

#	原则	说明
1	单向数据流	`ApiClient → Repository → BLoC/Cubit → Page/Widget` ，禁止反向
2	**domain 零依赖**	`domain/` 下不得 import `package:flutter` 或 `package:dio`
3	**presentation 不 import data**	BLoC/Cubit 通过构造函数注入 abstract Repository
4	常量集中管理	任何路径、阈值、魔法数字必须放 `core/constants/`
5	**freezed 四态 State**	所有 BLoC/Cubit 状态必须为 initial / loading / loaded / error
6	**Material 3 语义色**	通过 `Theme.of(context).colorScheme.*` 获取，禁止硬编码

二、环境搭建与工程配置

2.1 前置工具链

```
# Flutter SDK（3.x stable channel）
flutter --version    # Flutter 3.x · Dart 3.x

# 验证环境
flutter doctor -v

# HarmonyOS 工具链（可选，部署阶段需要）
# 参考 https://developer.huawei.com/consumer/cn/doc/harmonyos-guides-V5
```

2.2 本地开发启动

```
# 进入 Flutter 项目目录
cd flutter_app

# 安装依赖
flutter pub get

# 运行代码生成（freezed / json_serializable）
dart run build_runner build --delete-conflicting-outputs

# iOS 模拟器启动
flutter run -d ios

# Android 模拟器启动
flutter run -d android

# 指定设备启动
flutter devices            # 列出可用设备
flutter run -d <device_id>
```

2.3 关键配置文件

文件	作用	注意事项
`pubspec.yaml`	依赖管理 + 资源声明	版本约束使用 `^` 锁定主版本
`analysis_options.yaml`	静态分析规则	启用 `strict-casts` / `strict-raw-types` / `strict-inference`
`lib/core/config/app_config.dart`	运行时配置	从 `.env` 读取（flutter_dotenv）
`.env` / `.env.dev`	环境变量	`API_BASE_URL` 、 `FLUTTER_USE_MOCK` 等
`l10n.yaml`	国际化配置	当前仅中文，预留国际化结构

2.4 核心依赖清单

包名	用途	版本约束
`flutter_bloc`	状态管理（BLoC / Cubit）	^8.x
`freezed` + `freezed_annotation`	不可变数据类 + sealed union	^2.x
`json_serializable` + `json_annotation`	JSON 序列化	^6.x
`go_router`	声明式路由	^14.x
`dio`	HTTP 客户端	^5.x
`get_it`	依赖注入（Service Locator）	^7.x
`injectable` + `injectable_generator`	get_it 自动注册	^2.x
`intl`	日期 / 数字格式化	^0.19.x
`flutter_dotenv`	环境变量	^5.x
`equatable`	值相等比较（可选，freezed 已含）	^2.x
`bloc_test`	BLoC 单元测试	^9.x
`mocktail`	Mock 库	^1.x

三、目录结构详解

```
flutter_app/lib/
├── core/                          # 全局基础设施（跨 feature 共享）
│   ├── api/
│   │   ├── api_client.dart        # dio 封装: apiGet/apiPost/apiPatch/apiDelete
│   │   ├── api_exception.dart     # ApiException(code, message, statusCode)
│   │   ├── api_paths.dart         # API 路径常量
│   │   └── mock/                  # Mock 拦截器（开发环境）
│   │       ├── mock_interceptor.dart
│   │       └── modules/           # 按业务域拆分 mock 数据
│   │
│   ├── constants/
│   │   ├── business_rules.dart    # 业务规则常量（预警天数、逾期节点等）
│   │   └── ui_constants.dart      # UI 常量（分页大小、动画时长等）
│   │
│   ├── theme/
│   │   ├── app_theme.dart         # Material 3 ThemeData + ColorScheme
│   │   └── custom_colors.dart     # ThemeExtension<CustomColors> 语义扩展色
│   │
│   ├── router/
│   │   ├── app_router.dart        # go_router 路由表 + 守卫
│   │   └── route_paths.dart       # 路由路径常量
```

```
│   │
│   ├── di/
│   │   └── injection.dart            # get_it 依赖注入注册
│   │
│   └── utils/
│       ├── date_utils.dart           # 日期格式化工具
│       └── currency_utils.dart       # 金额格式化工具
│
├── features/                         # 按业务模块划分
│   ├── auth/                         # 认证模块
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   └── repositories/         # abstract class AuthRepository
│   │   ├── data/
│   │   │   ├── models/               # freezed DTO + fromJson
│   │   │   └── repositories/         # AuthRepositoryImpl（调用 ApiClient）
│   │   └── presentation/
│   │       ├── bloc/                 # AuthCubit + AuthState
│   │       └── pages/               # LoginPage
│   │
│   ├── dashboard/                    # 总览仪表盘
│   ├── assets/                       # M1 资产与空间
│   ├── contracts/                    # M2 合同管理
│   ├── finance/                      # M3 财务与 NOI
│   ├── workorders/                   # M4 工单系统
│   └── subleases/                    # M5 二房东穿透
│
├── shared/
│   ├── widgets/                      # 全局共享 Widget
│   │   ├── status_tag.dart           # 状态标签（语义色映射）
│   │   ├── empty_state.dart          # 空状态占位
│   │   ├── error_block.dart          # 错误提示块
│   │   └── paginated_list.dart       # 分页列表 + 下拉刷新
│   │
│   └── utils/                        # 工具函数
│       └── validators.dart           # 通用校验
│
└── main.dart                         # 入口（DI 注册 + runApp）
```

3.1 命名规范

类别	规范	示例
文件夹	`snake_case`	`api_client` , `floor_plan`
Dart 文件	`snake_case.dart`	`contract_list_cubit.dart`
类名	`PascalCase`	`ContractListCubit` , `ApiClient`
变量/函数	`camelCase`	`fetchList` , `isLoggedIn`
常量（顶层）	`camelCase` 或 `lowerCamelCase`	`defaultPageSize` , `apiContracts`
常量（类静态）	`camelCase`	`ApiPaths.contracts`
私有成员	`_camelCase`	`_repository` , `_dio`

四、核心分层架构与数据流

4.1 严格单向数据流

```
用户交互 → Widget 调 context.read<XxxCubit>().action()
         → Cubit 调 Repository 接口方法
         → RepositoryImpl 调 ApiClient
         → ApiClient 发 HTTP
                              ↓
用户看到 ← BlocBuilder 读 State ← Cubit emit 新 State ← Repository 返回数据
```

铁律:

层级	可以做	禁止做
Page/Widget	读 BLoC State、调 Cubit action、触发导航	直接调 ApiClient、import data 层
BLoC/Cubit	调 domain Repository 方法、emit State	import `package:flutter` 、import data 层
Domain	定义 Entity、abstract Repository、UseCase	import `package:flutter` 、import `package:dio`
Data	实现 Repository、调 ApiClient	包含 UI 逻辑、直接 emit State
ApiClient	管理 Token、解析信封、转换错误	包含业务逻辑

4.2 创建完整功能模块的文件依赖顺序

```
1. domain/entities/xxx.dart            ← 定义实体（纯 Dart，无依赖）
2. domain/repositories/xxx_repo.dart   ← 定义抽象 Repository 接口
3. core/api/api_paths.dart             ← 添加 API 路径常量
```

```
4. data/models/xxx_model.dart          ← freezed DTO + fromJson/toJson
5. data/repositories/xxx_repo_impl.dart ← Repository 实现（依赖 ApiClient + 3 + 4）
6. presentation/bloc/xxx_cubit.dart    ← Cubit（依赖 2）
7. presentation/bloc/xxx_state.dart    ← freezed 四态 State
8. presentation/pages/xxx_page.dart    ← 页面（依赖 6）
9. presentation/widgets/xxx_card.dart ← 子组件（依赖 1）
10. core/di/injection.dart             ← 注册 DI
11. core/router/app_router.dart        ← 注册路由
```

五、API 层开发规范

5.1 HTTP 客户端（api_client.dart）

客户端基于 `dio` 封装，已实现以下能力：

能力	实现
JWT 自动注入	请求拦截器从安全存储读取 `access_token`
信封自动解析	`apiGet<T>` 返回解封后的 `T`，不是原始 `Response`
401 自动刷新	响应拦截器检测 401 → 调 `refresh` → 重发排队请求
错误统一转换	所有错误抛出 `ApiException(code, message, statusCode)`
超时配置	默认 connectTimeout: 10s，receiveTimeout: 15s

已封装的方法签名：

```dart
class ApiClient {
  final Dio _dio;

  /// 获取单对象（自动解封 data 字段）
  Future<T> apiGet<T>(String path, {
    Map<String, dynamic>? queryParams,
    T Function(dynamic)? fromJson,
  });

  /// 获取分页列表（返回 items + meta）
  Future<ApiListResponse<T>> apiGetList<T>(String path, {
    Map<String, dynamic>? queryParams,
    required T Function(dynamic) fromJson,
  });

  /// 创建资源
  Future<T> apiPost<T>(String path, {
    dynamic data,
    T Function(dynamic)? fromJson,
  });
```

```
  /// 局部更新
  Future<T> apiPatch<T>(String path, {
    dynamic data,
    T Function(dynamic)? fromJson,
  });

  /// 删除资源
  Future<void> apiDelete(String path);
}
```

5.2 响应信封解析

ApiClient 内部统一解包服务端信封：

```
// 服务端统一格式
// 成功：{ "data": <payload>, "meta": { "page": 1, "pageSize": 20, "total": 639 } }
// 失败：{ "error": { "code": "CONTRACT_NOT_FOUND", "message": "合同不存在" } }

// ApiClient 内部处理：
// - 单对象接口：提取 body['data'] 传给 fromJson
// - 列表接口：提取 body['data']（数组）+ body['meta']（分页）
// - 错误响应：DioException → ApiException(code, message, statusCode)
```

5.3 Repository 实现范式

每个业务模块在 `data/repositories/` 下实现 domain 层的抽象接口：

```
// features/contracts/data/repositories/contract_repository_impl.dart

class ContractRepositoryImpl implements ContractRepository {
  final ApiClient _apiClient;

  ContractRepositoryImpl(this._apiClient);

  @override
  Future<ApiListResponse<Contract>> getContracts({
    int page = 1,
    int pageSize = 20,
    String? status,
    String? keyword,
  }) async {
    return _apiClient.apiGetList<Contract>(
      ApiPaths.contracts,
      queryParams: {
        'page': page,
        'pageSize': pageSize,
        if (status != null) 'status': status,
        if (keyword != null) 'keyword': keyword,
      },
      fromJson: (json) => ContractModel.fromJson(json as Map<String, dynamic>).toEntity(),
```

```
    );
  }

  @override
  Future<ContractDetail> getContractDetail(String id) async {
    return _apiClient.apiGet<ContractDetail>(
      '${ApiPaths.contracts}/$id',
      fromJson: (json) => ContractDetailModel.fromJson(json as Map<String,
dynamic>).toEntity(),
    );
  }
}
```

5.4 禁止事项

#	禁止	正确做法
1	在 BLoC/Widget 中直接 `Dio().get(...)`	通过 Repository → ApiClient
2	硬编码路径 `'/api/contracts'`	从 `ApiPaths.contracts` 导入
3	在 Repository 实现中写业务逻辑	业务逻辑放 UseCase 或 BLoC
4	透传 `DioException` 给 BLoC	ApiClient 统一转为 `ApiException`
5	在 ApiClient 中写业务判断	只做 HTTP + 信封解析 + 错误转换

六、BLoC / Cubit 状态管理规范

6.1 State 四态（必须使用 freezed sealed union）

```
// features/contracts/presentation/bloc/contract_list_state.dart

@freezed
sealed class ContractListState with _$ContractListState {
  const factory ContractListState.initial() = _Initial;
  const factory ContractListState.loading() = _Loading;
  const factory ContractListState.loaded(
    List<Contract> items, {
    PaginationMeta? meta,
  }) = _Loaded;
  const factory ContractListState.error(String message) = _Error;
}
```

6.2 Cubit 标准模板

```
// features/contracts/presentation/bloc/contract_list_cubit.dart
```

```
class ContractListCubit extends Cubit<ContractListState> {
  final ContractRepository _repository;

  ContractListCubit(this._repository)
      : super(const ContractListState.initial());

  Future<void> fetchList({int page = 1, int pageSize = defaultPageSize}) async {
    emit(const ContractListState.loading());
    try {
      final result = await _repository.getContracts(
        page: page,
        pageSize: pageSize,
      );
      emit(ContractListState.loaded(result.items, meta: result.meta));
    } catch (e) {
      emit(ContractListState.error(
        e is ApiException ? e.message : '加载失败，请重试',
      ));
    }
  }
}
```

6.3 何时选 Cubit vs Bloc

场景	选择	原因
简单 CRUD 列表	**Cubit**	方法调用即可，无需事件溯源
表单提交 + 校验	**Cubit**	逻辑线性，直接 emit
复杂状态机（合同审批流）	**Bloc**	需要事件日志、事件变换
搜索防抖 + 取消	**Bloc**	利用 `EventTransformer` 做 debounce

6.4 Bloc 事件驱动示例

```
@freezed
sealed class ContractSearchEvent with _$ContractSearchEvent {
  const factory ContractSearchEvent.queryChanged(String query) = _QueryChanged;
  const factory ContractSearchEvent.filterChanged(String? status) = _FilterChanged;
}

class ContractSearchBloc extends Bloc<ContractSearchEvent, ContractListState> {
  final ContractRepository _repository;

  ContractSearchBloc(this._repository) : super(const ContractListState.initial()) {
    on<_QueryChanged>(
      _onQueryChanged,
      transformer: debounce(const Duration(milliseconds: 300)),
    );
    on<_FilterChanged>(_onFilterChanged);
  }
```

```
  Future<void> _onQueryChanged(_QueryChanged event, Emitter<ContractListState> emit) async {
    emit(const ContractListState.loading());
    try {
      final result = await _repository.getContracts(keyword: event.query);
      emit(ContractListState.loaded(result.items, meta: result.meta));
    } catch (e) {
      emit(ContractListState.error(e is ApiException ? e.message : '搜索失败'));
    }
  }

  Future<void> _onFilterChanged(_FilterChanged event, Emitter<ContractListState> emit) async {
    emit(const ContractListState.loading());
    try {
      final result = await _repository.getContracts(status: event.status);
      emit(ContractListState.loaded(result.items, meta: result.meta));
    } catch (e) {
      emit(ContractListState.error(e is ApiException ? e.message : '筛选失败'));
    }
  }
}
```

6.5 禁止事项

#	禁止	正确做法
1	`import 'package:flutter/material.dart'`	BLoC 纯 Dart，不依赖 Flutter
2	`import '../data/...'`	通过构造函数注入 domain 层接口
3	`emit(XxxState.error(e.toString()))`	`e is ApiException ? e.message : '操作失败'`
4	`DateTime.now()`	注入 `Clock` 接口（`package:clock`）
5	BLoC 超过 200 行	按子领域拆分 Cubit

七、Page / Widget 层开发规范

7.1 Page 标准结构

```
// features/contracts/presentation/pages/contract_list_page.dart

class ContractListPage extends StatelessWidget {
  const ContractListPage({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (_) => getIt<ContractListCubit>()..fetchList(),
```

```
      child: Scaffold(
        appBar: AppBar(title: const Text('合同管理')),
        body: BlocBuilder<ContractListCubit, ContractListState>(
          builder: (context, state) => switch (state) {
            ContractListState.initial()  => const SizedBox.shrink(),
            ContractListState.loading()  => const Center(child: CircularProgressIndicator()),
            ContractListState.loaded(:final items, :final meta) =>
              _ContractListView(items: items, meta: meta),
            ContractListState.error(:final message) =>
              ErrorBlock(message: message, onRetry: () =>
context.read<ContractListCubit>().fetchList()),
          },
        ),
      ),
    );
  }
}
```

7.2 状态渲染规则

必须使用 Dart 3 `switch` expression 或 `.when()` 处理四态：

```
// ✅ 正确：switch expression（推荐）
builder: (context, state) => switch (state) {
  ContractListState.initial()  => const SizedBox.shrink(),
  ContractListState.loading()  => const LoadingIndicator(),
  ContractListState.loaded(:final items) => ListView(...),
  ContractListState.error(:final message) => ErrorBlock(message: message),
},

// ✅ 正确：.when() 方法（freezed 生成）
state.when(
  initial: () => const SizedBox.shrink(),
  loading: () => const LoadingIndicator(),
  loaded: (items, meta) => ListView(...),
  error: (message) => ErrorBlock(message: message),
),

// ❌ 禁止：散落的 if-is 判断
if (state is _Loading) return LoadingIndicator();
if (state is _Loaded) return ListView(...);
```

7.3 Page 与 Widget 职责边界

场景	Page 负责	Widget 负责
创建 BLoC	`BlocProvider(create: ...)`	不创建 BLoC
触发数据获取	`..fetchList()`	可通过回调触发刷新
导航跳转	`context.go(...)` / `context.push(...)`	回调通知 Page（`onTap`）
SnackBar/Dialog	`BlocListener` 内触发	不直接显示
状态读取	`BlocBuilder`	通过 props 接收数据

7.4 文件复杂度控制

信号	阈值	拆分策略
Page 文件行数	> 150 行	子区域提取到 `widgets/` 下独立 Widget
Widget 文件行数	> 100 行	继续拆分为更小组合 Widget
build 方法	> 60 行	提取私有 _buildXxx 方法或独立 Widget
Widget 树嵌套	> 4 层	提取内层为独立组件

7.5 日期显示规则

```
// ✅ 正确：统一用 intl 包格式化
import 'package:intl/intl.dart';

Text(DateFormat('yyyy-MM-dd').format(contract.startDate.toLocal()))

// ❌ 禁止：直接 toString 或手动拼接
Text(contract.startDate.toString())
Text('${date.year}-${date.month}-${date.day}')
```

八、实体与 DTO 类型体系

8.1 Domain Entity（纯 Dart）

```
// features/contracts/domain/entities/contract.dart

class Contract {
  final String id;
  final String contractNo;
  final String tenantName;
```

```
  final String unitNo;
  final ContractStatus status;
  final DateTime startDate;
  final DateTime endDate;
  final int monthlyRentCents; // 金额统一用分（int）

  const Contract({
    required this.id,
    required this.contractNo,
    required this.tenantName,
    required this.unitNo,
    required this.status,
    required this.startDate,
    required this.endDate,
    required this.monthlyRentCents,
  });
}

enum ContractStatus {
  draft,
  pendingApproval,
  active,
  expiringSoon,
  expired,
  terminated,
  renewed,
}
```

8.2 Data Model（freezed DTO）

```
// features/contracts/data/models/contract_model.dart

@freezed
class ContractModel with _$ContractModel {
  const factory ContractModel({
    required String id,
    required String contractNo,
    required String tenantName,
    required String unitNo,
    required String status,
    required String startDate,
    required String endDate,
    required int monthlyRentCents,
  }) = _ContractModel;

  factory ContractModel.fromJson(Map<String, dynamic> json) =>
      _$ContractModelFromJson(json);
}

extension ContractModelX on ContractModel {
  Contract toEntity() => Contract(
    id: id,
    contractNo: contractNo,
```

```
    tenantName: tenantName,
    unitNo: unitNo,
    status: ContractStatus.values.firstWhere(
      (e) => e.name == status,
      orElse: () => ContractStatus.draft,
    ),
    startDate: DateTime.parse(startDate),
    endDate: DateTime.parse(endDate),
    monthlyRentCents: monthlyRentCents,
  );
}
```

8.3 类型规则

规则	说明
Entity 纯 Dart	不依赖 `json_annotation`，无 `fromJson`
DTO 用 freezed	负责 JSON 序列化/反序列化
DTO → Entity 转换	`toEntity()` 扩展方法，在 Repository 实现中调用
金额用 `int`（分）	避免浮点精度问题；展示时 `cents / 100`
日期字段	API 传输 `String`（ISO 8601），Entity 用 `DateTime`
枚举	Entity 用 Dart enum，DTO 用 `String` + 解析

九、路由与导航（go_router）

9.1 路由表定义

```
// core/router/app_router.dart

final GoRouter appRouter = GoRouter(
  initialLocation: RoutePaths.dashboard,
  redirect: _authGuard,
  routes: [
    GoRoute(
      path: RoutePaths.login,
      builder: (_, __) => const LoginPage(),
    ),
    ShellRoute(
      builder: (_, __, child) => AppShell(child: child),
      routes: [
        GoRoute(
          path: RoutePaths.dashboard,
          builder: (_, __) => const DashboardPage(),
        ),
```

```
        GoRoute(
          path: RoutePaths.assets,
          builder: (_, __) => const AssetListPage(),
          routes: [
            GoRoute(
              path: 'buildings/:buildingId',
              builder: (_, state) => BuildingDetailPage(
                buildingId: state.pathParameters['buildingId']!,
              ),
            ),
            GoRoute(
              path: 'floors/:floorId',
              builder: (_, state) => FloorPlanPage(
                floorId: state.pathParameters['floorId']!,
              ),
            ),
          ],
        ),
        GoRoute(
          path: RoutePaths.contracts,
          builder: (_, __) => const ContractListPage(),
          routes: [
            GoRoute(
              path: ':id',
              builder: (_, state) => ContractDetailPage(
                contractId: state.pathParameters['id']!,
              ),
            ),
          ],
        ),
        // ... workorders, finance, subleases 类似结构
      ],
    ),
  ],
);
```

9.2 路由路径常量

```
// core/router/route_paths.dart

abstract final class RoutePaths {
  static const login = '/login';
  static const dashboard = '/dashboard';
  static const assets = '/assets';
  static const contracts = '/contracts';
  static const workorders = '/workorders';
  static const finance = '/finance';
  static const subleases = '/subleases';
  static const notifications = '/notifications';
  static const approvals = '/approvals';
  static const settings = '/settings';
}
```

9.3 导航方式

```
// ✅ 正确：使用 go_router
context.go(RoutePaths.dashboard);                        // 替换当前路由栈
context.push('${RoutePaths.contracts}/${contract.id}'); // 入栈新页面
context.pop();                                           // 返回上一页

// ❌ 禁止
Navigator.of(context).push(MaterialPageRoute(...));  // 不使用命令式导航
Navigator.pushNamed(context, '/contracts');           // 不使用 Named Routes
```

9.4 路由守卫

```
FutureOr<String?> _authGuard(BuildContext context, GoRouterState state) {
  final isLoggedIn = getIt<AuthRepository>().isLoggedIn;
  final isLoginRoute = state.matchedLocation == RoutePaths.login;

  if (!isLoggedIn && !isLoginRoute) return RoutePaths.login;
  if (isLoggedIn && isLoginRoute) return RoutePaths.dashboard;
  return null; // 不重定向
}
```

十、依赖注入（get_it）

10.1 注册模式

```
// core/di/injection.dart

final getIt = GetIt.instance;

void configureDependencies() {
  // ── 基础设施 ──
  getIt.registerLazySingleton<Dio>(() => Dio(BaseOptions(
    baseUrl: dotenv.env['API_BASE_URL'] ?? 'http://localhost:8080',
    connectTimeout: const Duration(seconds: 10),
    receiveTimeout: const Duration(seconds: 15),
  )));

  getIt.registerLazySingleton<ApiClient>(() => ApiClient(getIt<Dio>()));

  // ── Repository（单例）──
  getIt.registerLazySingleton<ContractRepository>(
    () => ContractRepositoryImpl(getIt<ApiClient>()),
  );

  // ── BLoC / Cubit（每次新建实例）──
  getIt.registerFactory<ContractListCubit>(
```

```
    () => ContractListCubit(getIt<ContractRepository>()),
  );
}
```

10.2 注册规则

类型	注册方式	原因
ApiClient / Dio	`registerLazySingleton`	全局单例，共享连接池和拦截器
Repository	`registerLazySingleton`	无状态，单例即可
BLoC / Cubit	`registerFactory`	有状态，每个页面需要独立实例

十一、UI 规范与 Material 3 设计体系

11.1 ThemeData 配置

```
// core/theme/app_theme.dart

ThemeData buildAppTheme() => ThemeData(
  useMaterial3: true,
  colorScheme: ColorScheme.fromSeed(
    seedColor: const Color(0xFF1677FF), // 主品牌色
    brightness: Brightness.light,
  ),
  extensions: [customColors],
);
```

11.2 语义色扩展

```
// core/theme/custom_colors.dart

@immutable
class CustomColors extends ThemeExtension<CustomColors> {
  final Color success;    // leased / paid
  final Color warning;    // expiring_soon
  final Color danger;     // vacant / overdue
  final Color neutral;    // non_leasable / draft

  const CustomColors({
    required this.success,
    required this.warning,
    required this.danger,
    required this.neutral,
  });
```

```
  // ... copyWith / lerp 实现
}

const customColors = CustomColors(
  success: Color(0xFF52C41A),
  warning: Color(0xFFFAAD14),
  danger:  Color(0xFFFF4D4F),
  neutral: Color(0xFF8C8C8C),
);
```

11.3 状态色语义映射

业务状态	语义色	获取方式	示例场景
`leased` / `paid` / `active`	成功（绿）	`Theme.of(context).extension<CustomColors>()!.success`	已租 / 已核销
`expiring_soon` / `warning`	预警（黄/橙）	`...customColors.warning`	即将到期
`vacant` / `overdue` / `terminated`	危险（红）	`...customColors.danger`	空置 / 逾期
`non_leasable` / `draft`	中性（灰）	`...customColors.neutral`	非可租 / 草稿

11.4 颜色使用禁令

```
// ❌ 禁止：硬编码颜色
Text('已租', style: TextStyle(color: Colors.green))
Container(color: Color(0xFF52C41A))

// ✅ 正确：从 Theme 获取
final colors = Theme.of(context).extension<CustomColors>()!;
Text('已租', style: TextStyle(color: colors.success))

// ✅ 正确：使用 ColorScheme
Text('主标题', style: TextStyle(color: Theme.of(context).colorScheme.onSurface))
```

11.5 状态标签 Widget

```
// shared/widgets/status_tag.dart

class StatusTag extends StatelessWidget {
  final String status;
  final String? label;

  const StatusTag({super.key, required this.status, this.label});

  @override
  Widget build(BuildContext context) {
```

```
    final colors = Theme.of(context).extension<CustomColors>()!;
    final config = _statusConfig(status, colors);

    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 2),
      decoration: BoxDecoration(
        color: config.color.withOpacity(0.1),
        borderRadius: BorderRadius.circular(4),
      ),
      child: Text(
        label ?? config.label,
        style: TextStyle(color: config.color, fontSize: 12),
      ),
    );
  }

  static _StatusConfig _statusConfig(String status, CustomColors colors) =>
      switch (status) {
        'active' || 'leased' || 'paid' => _StatusConfig(colors.success, '生效中'),
        'expiring_soon' || 'pending'    => _StatusConfig(colors.warning, '即将到期'),
        'vacant' || 'overdue' || 'terminated' => _StatusConfig(colors.danger, '空置'),
        _ => _StatusConfig(colors.neutral, status),
      };
}

class _StatusConfig {
  final Color color;
  final String label;
  const _StatusConfig(this.color, this.label);
}
```

十二、Mock 数据层规则

12.1 控制开关

通过 `flutter_dotenv` 读取 `.env` 中 `FLUTTER_USE_MOCK=true/false`：

```
// 在 DI 初始化阶段
if (dotenv.env['FLUTTER_USE_MOCK'] == 'true') {
  getIt<Dio>().interceptors.add(MockInterceptor());
}
```

12.2 Mock 拦截器

```
// core/api/mock/mock_interceptor.dart

class MockInterceptor extends Interceptor {
  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler handler) {
```

```
    final mockData = _findMock(options.method, options.path);
    if (mockData != null) {
      handler.resolve(Response(
        requestOptions: options,
        statusCode: 200,
        data: mockData,
      ));
    } else {
      handler.next(options); // 未匹配则穿透到真实 API
    }
  }
}
```

12.3 规则

- 新增 API 模块时**必须同步创建对应 mock**
- Mock 数据返回完整信封格式 `{"data": ..., "meta": ...}`
- `FLUTTER_USE_MOCK=false` 时不加载 MockInterceptor
- 支持部分 mock + 部分真实混合（URL 未匹配则 fallthrough）

十三、常量管理

禁止在业务代码中硬编码任何魔法数字或字符串。

类型	归属文件	示例
业务规则常量	`core/constants/business_rules.dart`	预警天数 90/60/30、逾期节点 1/7/15 天
UI 展示常量	`core/constants/ui_constants.dart`	`defaultPageSize = 20` 、动画时长
API 路径常量	`core/api/api_paths.dart`	`ApiPaths.contracts = '/api/contracts'`
路由路径常量	`core/router/route_paths.dart`	`RoutePaths.dashboard = '/dashboard'`

十四、错误处理体系

14.1 三层错误处理链

```
dio DioException
     ↓ ApiClient 拦截
ApiException(code, message, statusCode)
     ↓ Repository 透传
BLoC catch
     ↓ emit
XxxState.error(friendlyMessage)
     ↓ BlocBuilder
```

```
ErrorBlock Widget 展示
```

14.2 ApiException 定义

```
// core/api/api_exception.dart

class ApiException implements Exception {
  final String code;     // 如 'CONTRACT_NOT_FOUND'
  final String message;  // 用户可读消息
  final int? statusCode; // HTTP 状态码

  const ApiException(this.code, this.message, [this.statusCode]);

  @override
  String toString() => 'ApiException($code: $message)';
}
```

14.3 BLoC 层统一 catch

```
// ✅ 正确
try {
  final result = await _repository.getContracts();
  emit(ContractListState.loaded(result.items));
} catch (e) {
  emit(ContractListState.error(
    e is ApiException ? e.message : '操作失败，请重试',
  ));
}

// ❌ 错误：透传原始异常
catch (e) {
  emit(ContractListState.error(e.toString())); // 可能泄漏内部信息
}
```

十五、平台适配（iOS / Android / HarmonyOS Next）

15.1 三平台策略

平台	适配要点
iOS	Safe Area、Cupertino 风格可选（Material 为主）
Android	Material 3 原生适配
HarmonyOS Next	Flutter 官方 HarmonyOS 支持 + 华为 DevEco 工具链

15.2 平台判断

```
import 'dart:io' show Platform;

// 仅在必须的平台差异场景使用
if (Platform.isIOS) {
  // iOS 特有逻辑（如 APNs 推送注册）
} else if (Platform.isAndroid) {
  // Android 特有逻辑（如 FCM 推送注册）
}
```

15.3 Safe Area

```
// ✅ 所有页面顶层使用 SafeArea
Scaffold(
  body: SafeArea(
    child: /* 页面内容 */,
  ),
)
```

十六、测试策略

16.1 测试金字塔

层级	工具	覆盖范围	必须覆盖
单元测试	`bloc_test` + `mocktail`	BLoC/Cubit 状态变迁	✅ 所有 BLoC
单元测试	`test`	Domain Entity / UseCase	✅ 核心计算
Widget 测试	`flutter_test`	页面渲染 + 交互	关键页面
集成测试	`integration_test`	端到端流程	核心业务流

16.2 BLoC 测试范式

```
// test/features/contracts/presentation/bloc/contract_list_cubit_test.dart

import 'package:bloc_test/bloc_test.dart';
import 'package:mocktail/mocktail.dart';

class MockContractRepository extends Mock implements ContractRepository {}

void main() {
  late MockContractRepository repository;
  late ContractListCubit cubit;

  setUp(() {
```

```
    repository = MockContractRepository();
    cubit = ContractListCubit(repository);
  });

  blocTest<ContractListCubit, ContractListState>(
    '成功获取合同列表',
    build: () {
      when(() => repository.getContracts(page: 1, pageSize: 20))
          .thenAnswer((_) async => ApiListResponse(
                items: [testContract],
                meta: const PaginationMeta(page: 1, pageSize: 20, total: 1),
              ));
      return cubit;
    },
    act: (cubit) => cubit.fetchList(),
    expect: () => [
      const ContractListState.loading(),
      isA<ContractListState>()
          .having((s) => s is _Loaded, 'is loaded', true),
    ],
  );

  blocTest<ContractListCubit, ContractListState>(
    'API 异常时emit error 状态',
    build: () {
      when(() => repository.getContracts(page: 1, pageSize: 20))
          .thenThrow(const ApiException('SERVER_ERROR', '服务器错误', 500));
      return cubit;
    },
    act: (cubit) => cubit.fetchList(),
    expect: () => [
      const ContractListState.loading(),
      const ContractListState.error('服务器错误'),
    ],
  );
}
```

十七、性能优化

优化点	实践
列表性能	使用 `ListView.builder`（惰性构建），避免 `ListView(children: [])`
图片缓存	使用 `cached_network_image` 包
const 构造	静态 Widget 加 `const` 前缀，减少重建
BLoC 选择性重建	`BlocSelector` 只监听需要的字段，避免全量重建
大列表	分页加载（`defaultPageSize = 20`），避免一次性加载全部
路由预加载	关键页面使用 `GoRoute.pageBuilder` 添加自定义过渡

十八、新功能模块开发 SOP

当需要新增一个业务模块（如 `features/xxx/`）时，按以下顺序执行：

Step 1：Domain 层

```
features/xxx/domain/
  entities/xxx.dart              ← 纯 Dart 实体类
  repositories/xxx_repository.dart ← abstract class（接口契约）
```

Step 2：Data 层

```
features/xxx/data/
  models/xxx_model.dart          ← @freezed DTO + fromJson + toEntity()
  repositories/xxx_repository_impl.dart ← 实现 domain 接口，调 ApiClient
```

Step 3：Presentation 层

```
features/xxx/presentation/
  bloc/xxx_cubit.dart            ← Cubit（注入 domain Repository）
  bloc/xxx_state.dart            ← @freezed 四态 State
  pages/xxx_list_page.dart       ← Page（BlocProvider + BlocBuilder）
  widgets/xxx_card.dart          ← 子组件
```

Step 4：注册集成

```
core/api/api_paths.dart          ← 添加 API 路径常量
core/di/injection.dart           ← 注册 Repository + Cubit
core/router/app_router.dart      ← 添加路由
core/router/route_paths.dart     ← 添加路径常量
```

Step 5：Mock + 测试

```
core/api/mock/modules/xxx_mock.dart  ← Mock 数据
test/features/xxx/                   ← bloc_test 单元测试
```

十九、常见问题与排错

Q1：freezed 代码生成报错

```
# 清理并重新生成
dart run build_runner clean
dart run build_runner build --delete-conflicting-outputs
```

Q2：get_it 注册找不到实例

- 确认 `configureDependencies()` 在 `main()` 中调用
- 确认注册顺序：先注册被依赖项（ApiClient → Repository → Cubit）

Q3：BlocBuilder 不刷新

- 确认 State 使用 `freezed`（保证值相等比较正确）
- 确认 `emit` 的是新对象（不是修改旧对象的字段）

Q4：go_router 路由不匹配

- 确认路径以 `/` 开头
- 子路由 path 不需要前导 `/`（相对路径）
- 检查 `redirect` 函数是否拦截了目标路由

Q5：HarmonyOS 编译失败

- 确认 Flutter SDK 版本支持 HarmonyOS
- 参考华为 DevEco 官方文档配置环境

二十、附录

附录 A：完整依赖清单（pubspec.yaml 节选）

```
dependencies:
  flutter:
    sdk: flutter
  flutter_bloc: ^8.1.0
  go_router: ^14.0.0
  dio: ^5.4.0
```

```
  get_it: ^7.6.0
  freezed_annotation: ^2.4.0
  json_annotation: ^4.8.0
  intl: ^0.19.0
  flutter_dotenv: ^5.1.0
  cached_network_image: ^3.3.0
  equatable: ^2.0.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.4.0
  freezed: ^2.4.0
  json_serializable: ^6.7.0
  bloc_test: ^9.1.0
  mocktail: ^1.0.0
  flutter_lints: ^3.0.0
```

附录 B：BLoC / Cubit 清单

模块	BLoC / Cubit	State 类型
认证	`AuthCubit`	`AuthState`
总览	`DashboardCubit`	`DashboardState`
资产	`AssetListCubit` / `BuildingDetailCubit` / `FloorPlanCubit`	各自四态 State
合同	`ContractListCubit` / `ContractDetailCubit` / `ContractFormCubit`	各自四态 State
财务	`FinanceOverviewCubit` / `InvoiceListCubit` / `KpiCubit`	各自四态 State
工单	`WorkorderListCubit` / `WorkorderDetailCubit` / `WorkorderFormCubit`	各自四态 State
二房东	`SubleaseListCubit` / `SubleaseDetailCubit`	各自四态 State
通知	`NotificationCubit`	`NotificationState`
审批	`ApprovalQueueCubit`	`ApprovalQueueState`

附录 C：与 Admin 端对照

维度	Flutter App	Admin PC
语言	Dart	TypeScript
UI 框架	Material 3	Element Plus
状态管理	flutter_bloc (BLoC/Cubit)	Pinia (setup)
HTTP	dio	axios
路由	go_router	Vue Router 4
DI	get_it	无（直接 import）
错误类型	`ApiException`	`ApiError`
状态色	`ThemeExtension<CustomColors>`	`el-tag type`